Econ 144 Project 3 - Complete Forecast Analysis of NYC Daily Attendance Data

Cedar Powers

Table of contents

I. Introduction

On the whole, it is difficult to find education time series at frequencies higher than annual. This greatly limits the potential for meaningful forecast analysis by members of the public. One exciting break from this mold is the New York City Open Data program. The Open Data program published hundred of large datasets relating to the New York City public. For this analysis I explore the 2018-2021 Daily Attendance by School dataset, which reports enrollment and attendance data for over 1600 K-12 schools over a four year period. I fit ARIMA, ETS, NNETAR, Prophet, ARFIMA, and Kalman Filter models in pursuit of a strong forecast combination that can give meaningful insight into future trends in New York City K-12 attendance. The data is recorded daily from 2018-2021 and this three school year term allows for analysis of the short to medium-term dynamics (seasonality, recent trend, possibly cycles) and some near-term forecasting. One notable challenge is that school year data is not reported during weekends or breaks from school; I address this with a simple ARMA interpolation to approximate a continuous time series.

II. Analysis and Results

Setup

```
rm(list=ls(all=TRUE))

# reproducibility
set.seed(144)

# core tidy packages
library(tidyverse)
library(fable)
library(tsibble)
library(feasts)
library(ggtime)
library(patchwork)
library(broom)

# additional packages
library(fable.prophet)
library(forecast)
library(stats)
library(imputeTS)
library(dlm)
```

The data for this analyis is meticulously sorted by school meaning there are ample oportunities to model attendance in specific schools or communities. For the sake of this general analysis, hovever, I will sum the variables into total NYC daily attendance across all schools. Further, to create a continuous series for effective analysis, I use ARMA(1,1) Kalman interpolation to fill in the weekend and school break gaps in the attendance data. This is primarily because there tend to be large spikes in absenteeism on the last days of a semester which spline or linear interpolation would greatly exagerate. The ARMA(1,1) model offers a good balance between maintaining the overall trend of the series during the school year and approaching zero (actual attendance) during summer break.

```
nyc_daily <- read.csv("2018-2021_Daily_Attendance_by_School_20260604.csv")

# remove schools with incomplete attendance information (107 rows out of 736578)
nyc_daily <- nyc_daily |>
    group_by(School.DBN) |>
    filter(Enrolled != "") |>
```

```
    ungroup()

# group by day, sum absent total, ungroup
nyc_daily <- nyc_daily |>
    group_by(Date) |>
    mutate(
        total_enrolled = sum(parse_number(Enrolled)),
        total_present = sum(parse_number(Present)),
        total_absent = sum(parse_number(Absent))
    ) |>
    select(Date, total_enrolled, total_present, total_absent) |>
    distinct() |>
    ungroup()

# normalize date column
nyc_daily <- nyc_daily |>
    mutate(Date = str_sub(Date, 1, 10)) |>
    mutate(Date = (parse_date_time(Date, orders = c("ymd", "mdy"))))

# add missing days
timeline <- tibble(
    "Date" = seq(ymd("2018-09-05"), ymd("2021-06-25"), by = "days")
)

nyc_daily <- nyc_daily |>
    full_join(timeline, by = "Date") |>
    mutate(Date = as_date(Date)) |>
    arrange(Date)

# interpolate the nonlinearity of school years
# using an ARMA(1,1) interpolation to best maintain series dynamics
nyc_daily <- nyc_daily |>
    mutate(
        total_enrolled = na_kalman(
            total_enrolled,
            model = arima(nyc_daily$total_enrolled, order = c(1, 0, 1))$model
        ),
        total_present = na_kalman(
            total_present,
            model = arima(nyc_daily$total_present, order = c(1, 0, 1))$model
        ),
        total_absent = na_kalman(
```

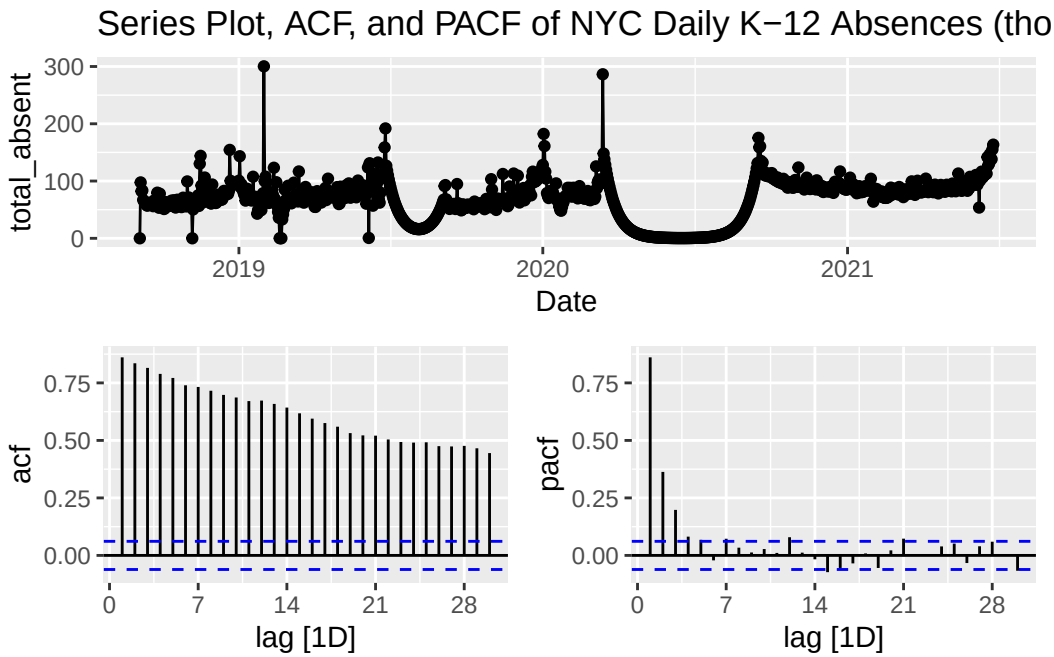
```
            total_absent,
            model = arima(nyc_daily$total_absent, order = c(1, 0, 1))$model
        )
    )

# transform to thousands of students
nyc_daily <- nyc_daily |>
    mutate(
        total_enrolled = (total_enrolled/ 1000),
        total_present = (total_present / 1000),
        total_absent = (total_absent / 1000)
    )

# write to file
write.csv(nyc_daily, "nyc_daily_interp_2018_21.csv", row.names = FALSE)
```

```
# analysis will focus on the total_absent variable
# ts_ for tsibble and ots_ for original time series object
ts_attend <- as_tsibble(nyc_daily, index = Date) |>
    select(Date, total_absent)
ots_attend <- as.ts(ts_attend)
```

Data Exploration

Series Plots

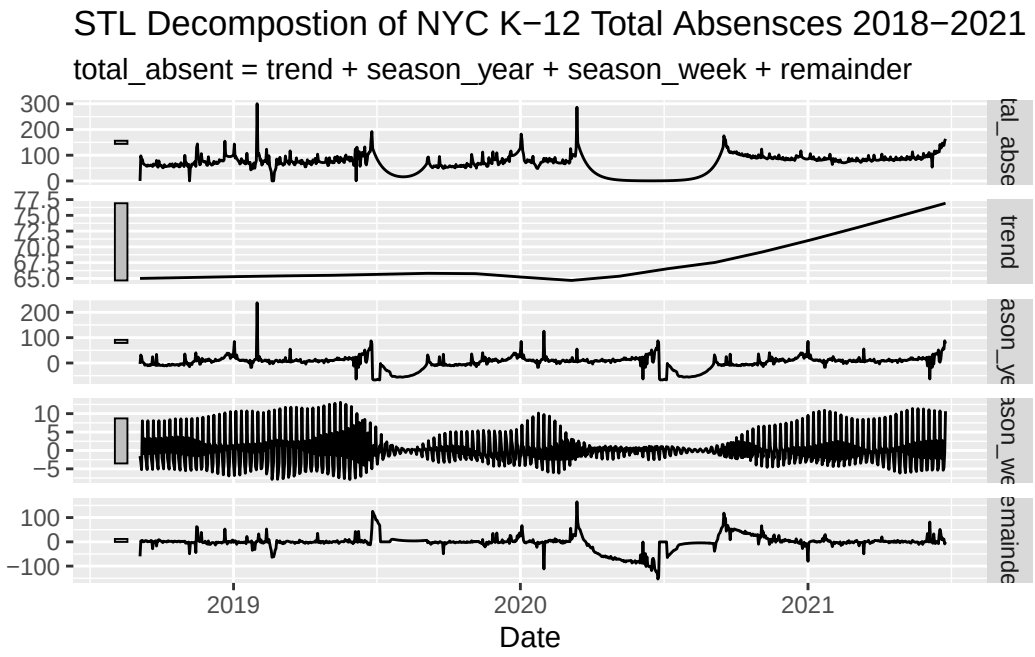
```
ts_attend |>
    gg_tsdisplay(total_absent, plot_type = "partial") +
    labs(title = "Series Plot, ACF, and PACF of NYC Daily K-12 Absences (thousands)")
```



In the absenteeism series plot we can see that the interpolation has primarily maintained the trend of the real data and filled in the two summer break gaps (one larger due to COVID-19) with a gradual decay towards zero. The series shows a slow positive linear trend, the spikes in the PACF and slow decay of the ACF indicate an AR(5) process with long memory.

STL Decomposition

```
ts_attend |>
    model(
        STL(total_absent ~ trend() + season(), robust = TRUE)
    ) |>
    components() |>
    autoplot() +
    labs(
        title = "STL Decompostion of NYC K-12 Total Absensces 2018-2021"
    )
```



The STL decompostion shows a slightly more quadratic trend with clear weekly seasonality and strong cycles representing the summer break dip. All characteristics are significant at a factor of 10 or greater.

Model Specification

Train/Test Split

```
train_ts_attend <- ts_attend |>
    filter(Date < mdy("3/25/2021")) # 3 months from end
test_ts_attend <- ts_attend |>
    filter(Date >= mdy("3/25/2021"))
train_ots_attend <- ots_attend[1:933]
test_ots_attend <- ots_attend[934:1026]
```

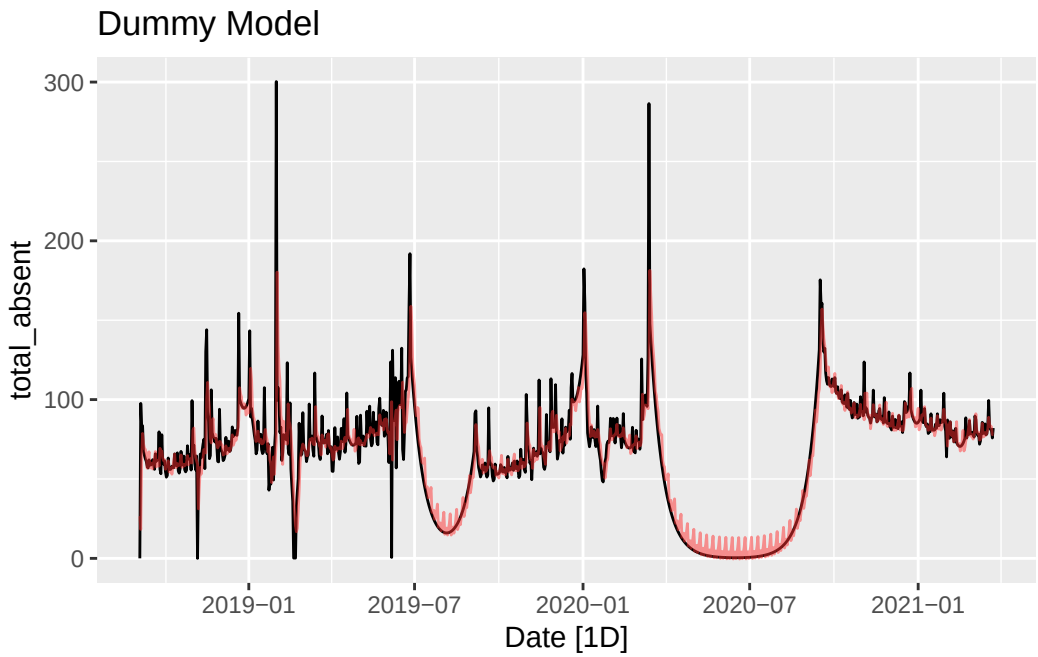
To ensure my models are not overfit to the data, I holdout the last three months for diagnostic testing.

Dummy Model

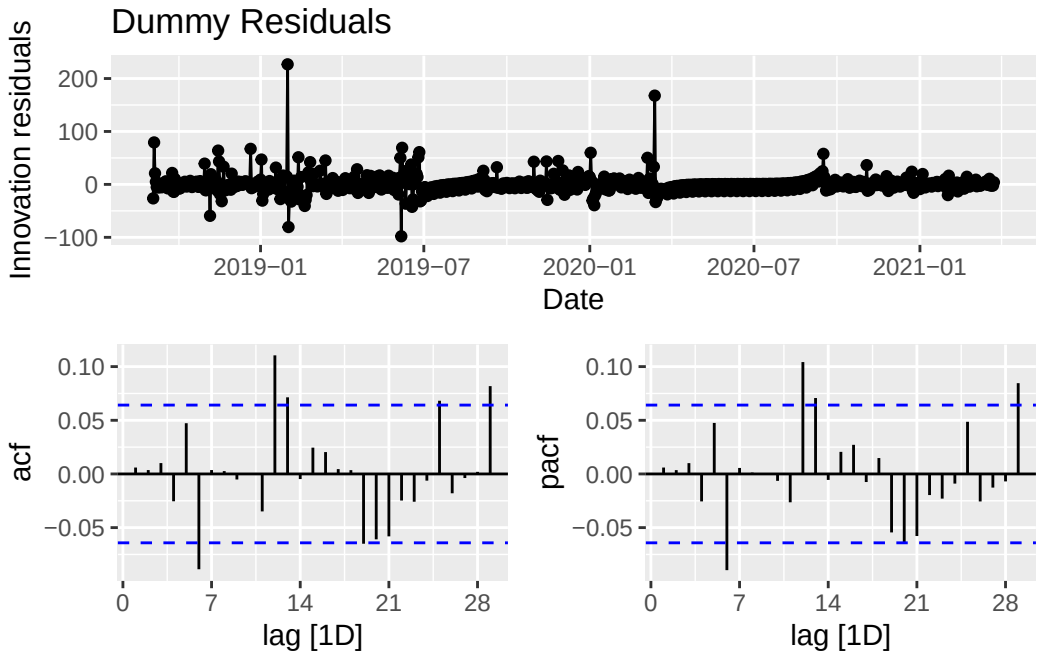
```r
# model with trend time dummy, seasonal dummies, and ARIMA errors
dummy_fit <- train_ts_attend |>
    model(
        dummy = ARIMA(total_absent ~ trend() + season())
    )

report(dummy_fit)
```

```
Series: total_absent
Model: LM w/ ARIMA(2,0,1)(0,0,1)[7] errors

Coefficients:
         ar1     ar2      ma1     sma1  trend()  season()week2  season()week3
      0.9487  0.0277  -0.5105  -0.0864   0.0069         6.8272        -1.7752
s.e.  0.0680  0.0644   0.0598   0.0346   0.0326         1.4358         1.4914
      season()week4  season()week5  season()week6  season()week7  intercept
             -2.324        -0.7172        -4.6048        -3.2169    62.5836
s.e.          1.527         1.5274         1.4894         1.4343    17.9765

sigma^2 estimated as 258.1:  log likelihood=-3909.41
AIC=7844.81   AICc=7845.21   BIC=7907.71
```

```r
# fit plot
dummy_afit <- augment(dummy_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        dummy_afit,
        color = "firebrick1",
        alpha = 0.5) +
    labs(title = "Dummy Model")
```




```
# residuals plot
dummy_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "Dummy Residuals")
```



The dummy model fits an ARIMA(2,0,1)(0,0,1) model for the errors with an AR(2), MA(1) and SMA(1). It fits weekly seasonality with a period of seven. The residuals are close to white noise and the fit appears decent, albeit noisy.

```
dummy_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 dummy     27.9    0.0147
```

A Ljung-Box test on the residuals fails to reject the null that residuals are correlated, indicating there are still some dynamics present, so the Dummy model alone is not a perfect fit.

ARIMA

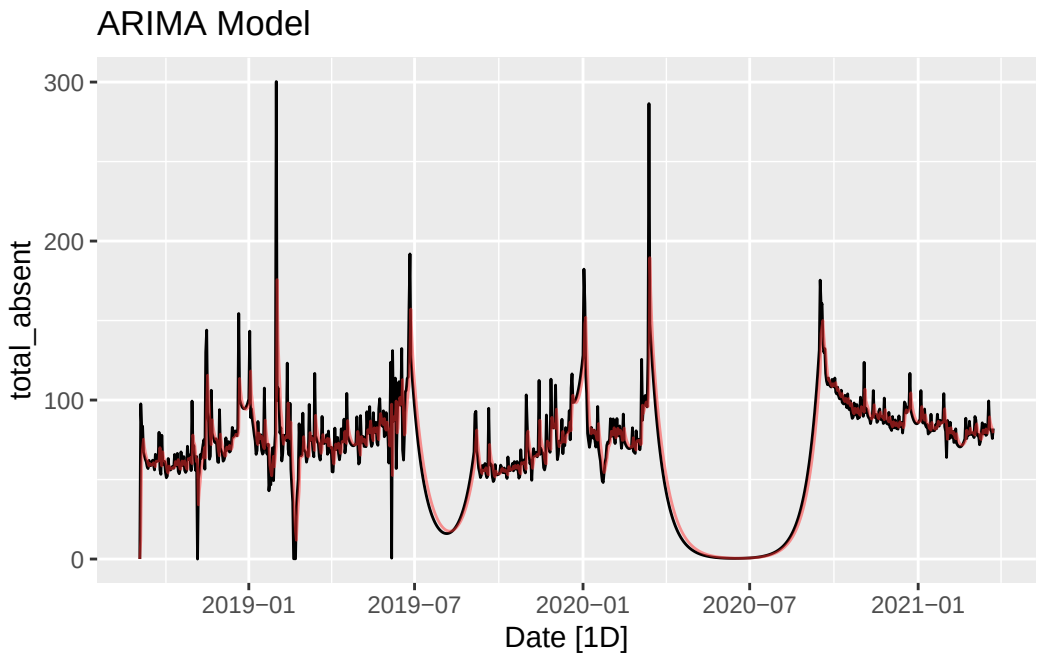
```
# automatic ARIMA modeling
arima_fit <- train_ts_attend |>
    model(
        arima = ARIMA(total_absent ~ pdq() + PDQ())
    )
```

```
Warning in sqrt(diag(best$var.coef)): NaNs produced
```

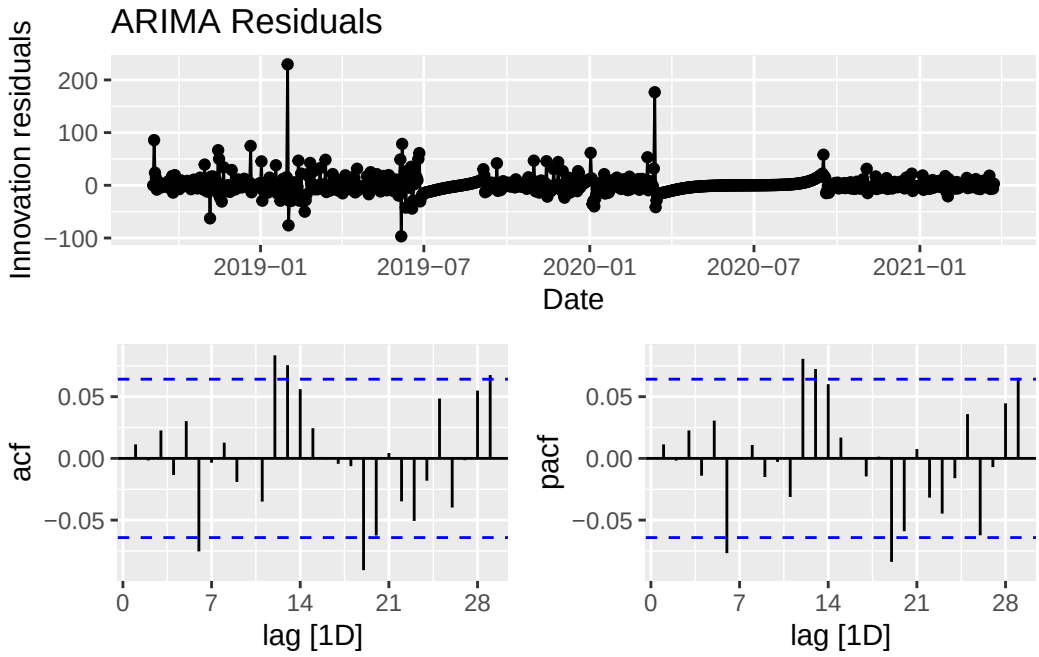
```
report(arima_fit)
```

```
Series: total_absent
Model: ARIMA(2,1,2)

Coefficients:
         ar1     ar2      ma1     ma2
      0.8766  0.0036  -1.4192  0.4557
s.e.     NaN     NaN      NaN     NaN

sigma^2 estimated as 276.4:  log likelihood=-3940.49
AIC=7890.98   AICc=7891.05   BIC=7915.17
```

```
# fit plot
arima_afit <- augment(arima_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        arima_afit,
        color = "firebrick1",
        alpha = 0.5) +
    labs(title = "ARIMA Model")
```




```
# residuals plot
arima_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "ARIMA Residuals")
```



The automatic ARIMA model fits an ARIMA(2,1,2) model which includes an AR(2) component, an MA(2) component, and one level of differencing. The residuals resemble white noise which indicates this model is a good fit. It may perform worse relative to the other models due to the high frequency of data, however.

```
arima_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 arima     23.6    0.0507
```

A Ljung-Box test on the residuals narrowly rejects the null that residuals are correlated, indicating the model is a strong fit but there might be some subtle dynamics left.

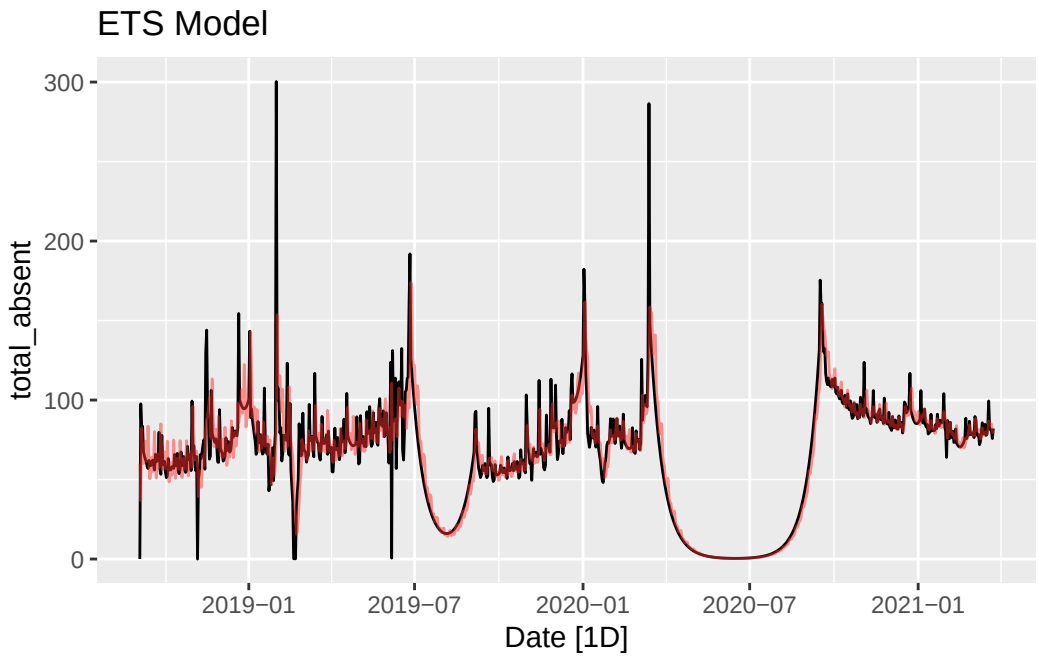
ETS

```
# automatic ETS modeling
ets_fit <- train_ts_attend |>
    model(
        ets = ETS(total_absent)
```

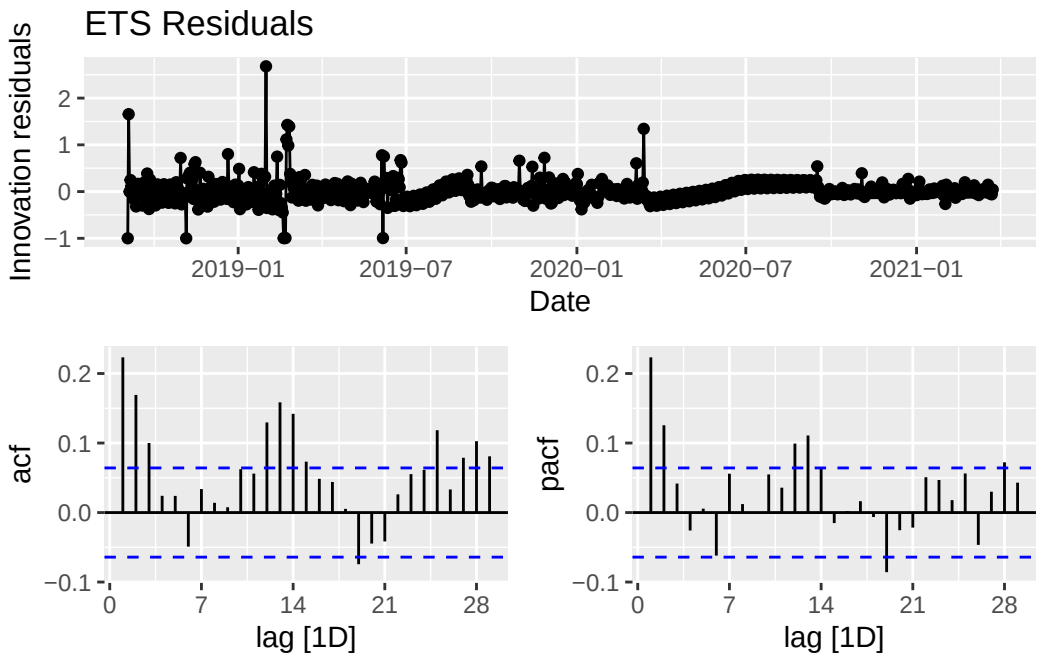
```
    )

report(ets_fit)
```

```
Series: total_absent
Model: ETS(M,Ad,M)
  Smoothing parameters:
    alpha = 0.3432856
    beta  = 0.005194202
    gamma = 0.02337942
    phi   = 0.94201

  Initial states:
     l[0]       b[0]     s[0]     s[-1]     s[-2]    s[-3]    s[-4]     s[-5]
 64.13404 -0.3834043 1.044061 0.9321444 0.8679407 1.046648 1.280202 0.8910028
     s[-6]
 0.9380011

  sigma^2:  0.054

     AIC     AICc      BIC
10681.40 10681.79 10744.29
```

```
# fit plot
ets_afit <- augment(ets_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        ets_afit,
        color = "firebrick1",
        alpha = 0.5) +
    labs(title = "ETS Model")
```




```
# residuals plot
ets_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "ETS Residuals")
```



The automatic ETS model fits an ETS(M,Ad,M) model with multiplicative errors, additive damped trend, and multiplicative seasonality. It is placing more weight on older level, trend, and seasonality values. This fit is weaker than the ARIMA model with some dynamics still present in the residuals. The ETS model struggles to capture short term patterns compared to ARIMA.

```
ets_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 ets       153.         0
```

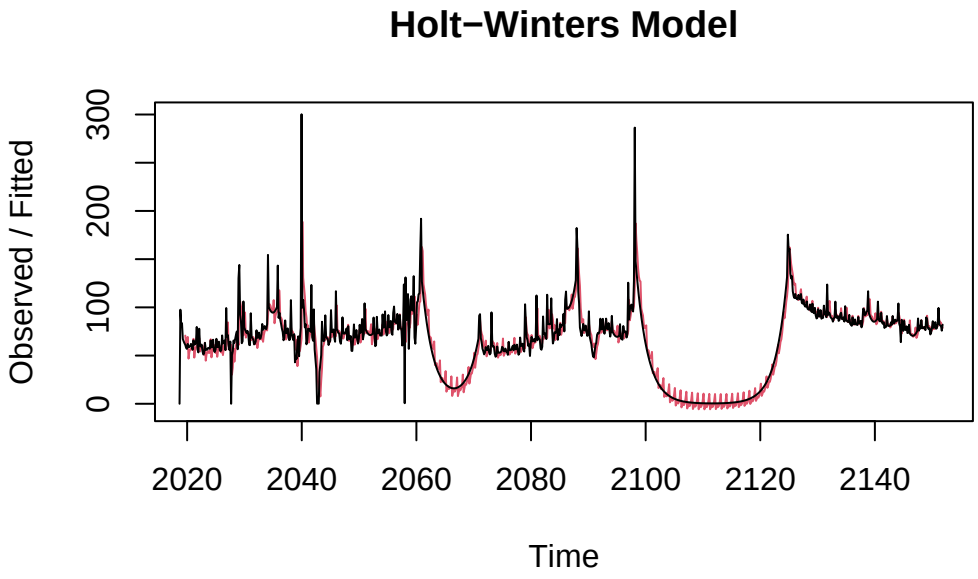
A Ljung-Box test on the residuals handedly fails to reject the null that residuals are correlated, as expected. This indicates there are still some dynamics present, and the ETS model alone is not a perfect fit.

Holt-Winters

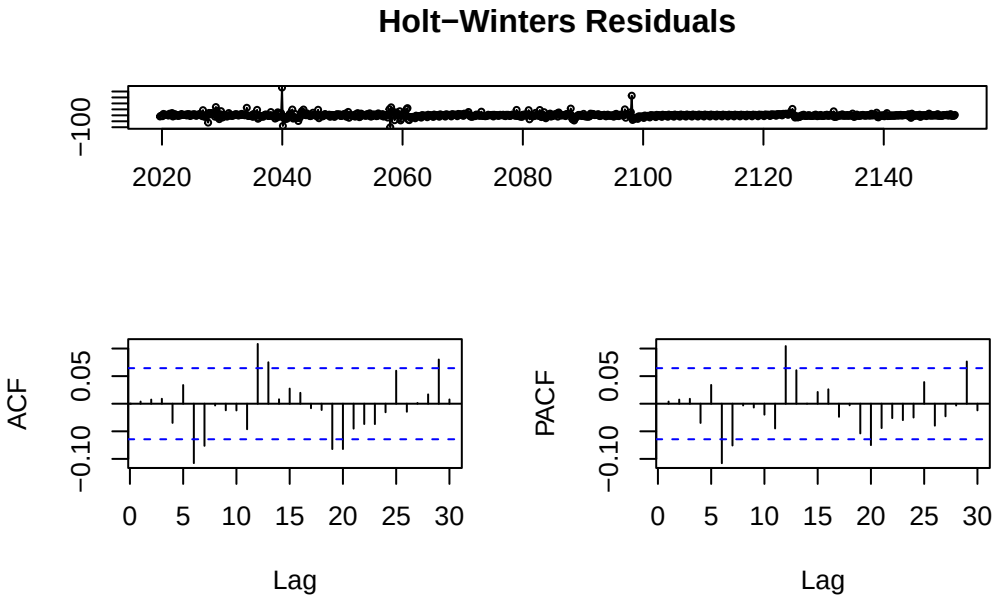
```r
# basic Holt-Winters model
hw_fit <- train_ts_attend |>
    HoltWinters()

hw_fit
```

```
Holt-Winters exponential smoothing with trend and additive seasonal component.

Call:
HoltWinters(x = train_ts_attend)

Smoothing parameters:
 alpha: 0.4625732
 beta : 0.005831283
 gamma: 0.02644665

Coefficients:
         [,1]
a  80.9138424
b   0.0542180
s1  1.0969810
s2  8.2243810
s3  0.4597869
s4 -0.1094350
s5  1.2929631
s6 -1.1926578
s7 -0.6363532
```

```r
# fit plot
plot(hw_fit, main = "Holt-Winters Model")
```




```r
# residuals plot
hw_resid <- resid(hw_fit)
tsdisplay(hw_resid, main = "Holt-Winters Residuals")
```



The Holt-Winters model fits a weekly seasonal period with seven terms and is putting more weight on older data across level, trend, and seasonality. The fit is decent but has some negative values during the summer dips. The residuals are close to white noise but have some subtle dynamics.

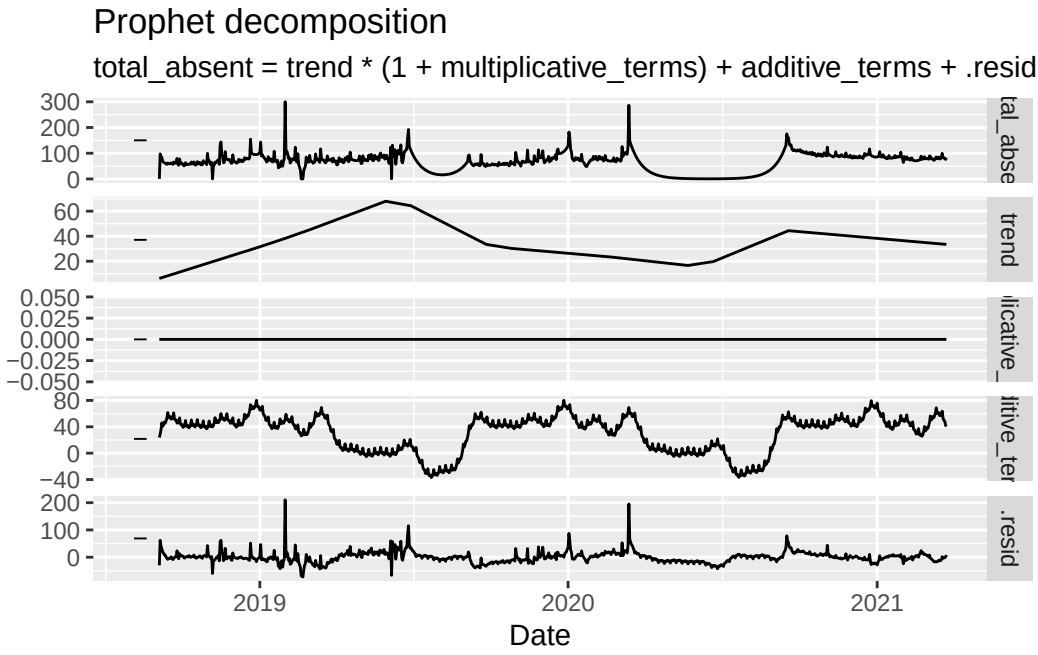
```
Box.test(hw_resid, type = "Ljung-Box", lag = 14)
```

```
    Box-Ljung test

data:  hw_resid
X-squared = 37.304, df = 14, p-value = 0.0006634
```

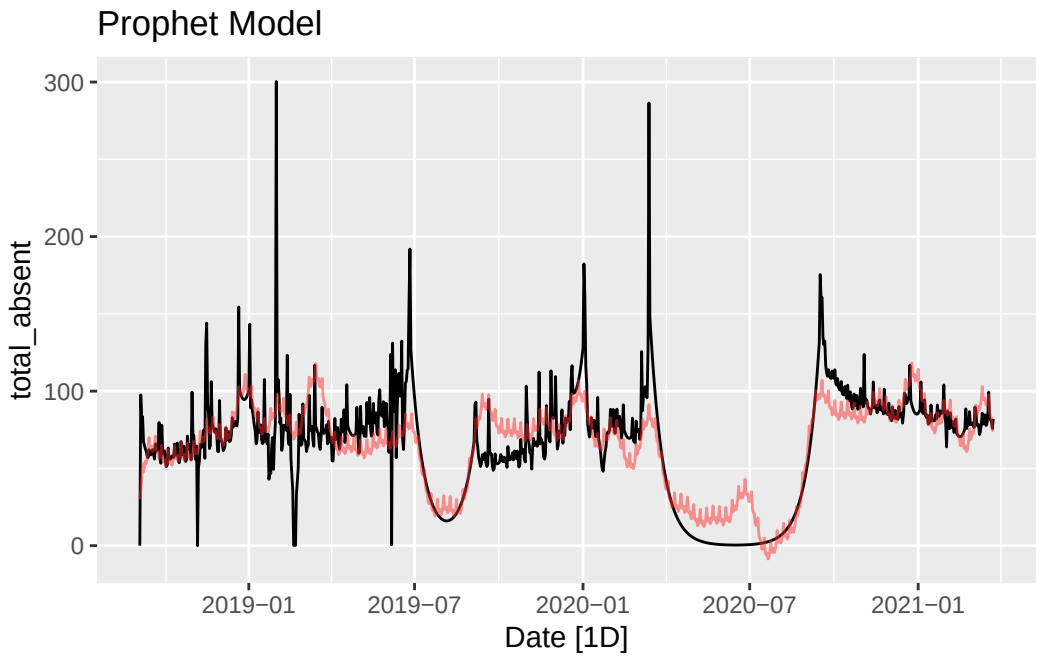
A Ljung-Box test on the residuals fails to reject the null that residuals are correlated, as expected indicating there are still some dynamics present, and the Holt-Winters model alone is not a perfect fit.

Prophet

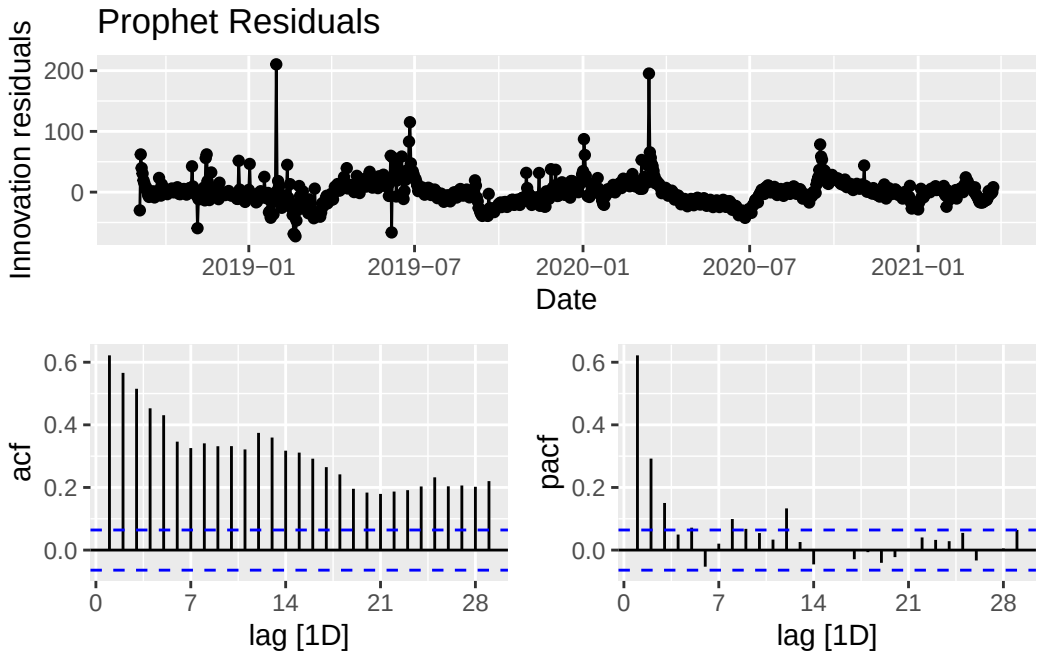
```
# automatic ETS modeling
pro_fit <- train_ts_attend |>
    model(
        prophet = prophet(total_absent)
    )

pro_fit |>
  components() |>
  autoplot()
```




```
# fit plot
pro_afit <- augment(pro_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        pro_afit,
        color = "firebrick1",
        alpha = 0.5) +
    labs(title = "Prophet Model")
```




```
# residuals plot
pro_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "Prophet Residuals")
```



At first glance, the Prophet model performs quite poorly on this data. It is not well fit to the training set and there are major dynamics left in the residuals. However, it fits daily, weekly, and yearly seasonality simultaneously and should account for the holiday effects to some extent so it might be promising in a combination or on the testing set.

```
pro_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model  lb_stat lb_pvalue
  <chr>     <dbl>     <dbl>
1 prophet   2258.         0
```

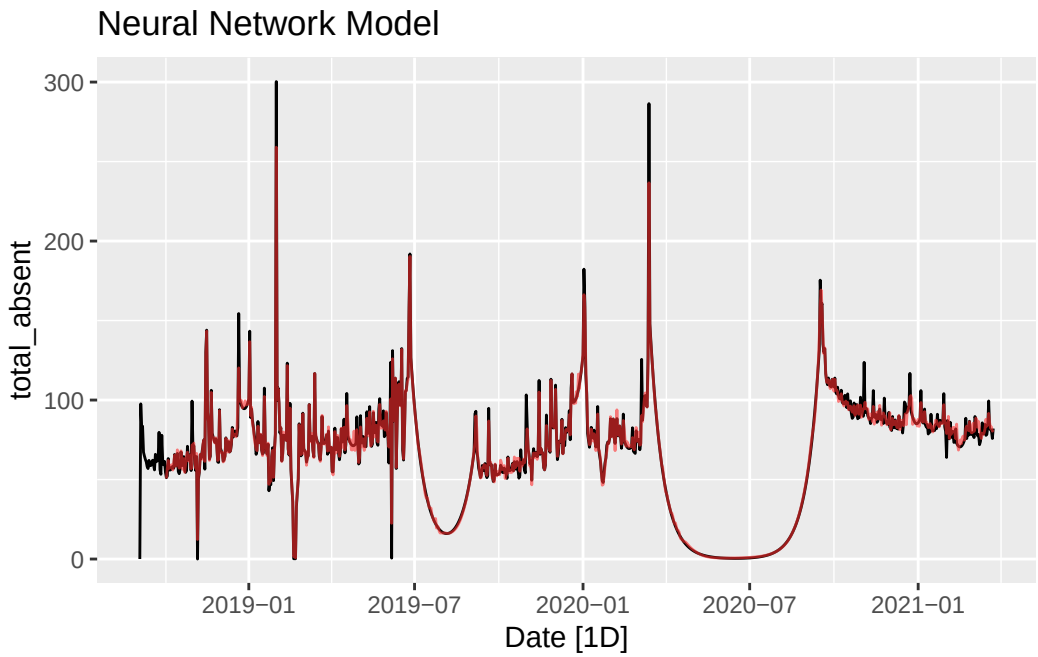
A Ljung-Box test on the residuals once again handedly fails to reject the null that residuals are correlated, as expected. This indicates there are still some dynamics present, and the Prophet model alone is not a perfect fit.

Neural Network Autoregressive

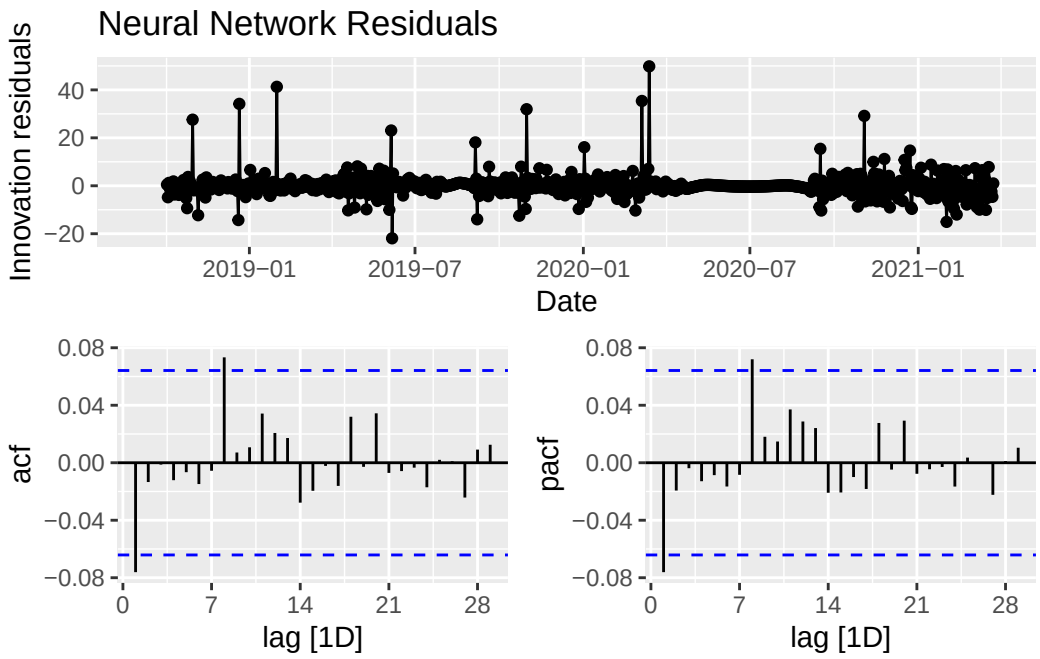
```r
set.seed(144)

nnet_fit <- train_ts_attend |>
    model(
        nnet = NNETAR(total_absent)
    )

report(nnet_fit)
```

```
Series: total_absent
Model: NNAR(29,1,15)[7]

Average of 20 networks, each of which is
a 29-15-1 network with 466 weights
options were - linear output units

sigma^2 estimated as 21.13
```

```r
# fit plot
nnet_afit <- augment(nnet_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        nnet_afit,
        color = "firebrick1",
        alpha = 0.6) +
    labs(title = "Neural Network Model")
```




```
# residuals plot
nnet_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "Neural Network Residuals")
```



The Neural Network Autoregressive model fits a daily seven period seasonal NNAR(29,1,15) which has 29 regular lags, 1 weekly seasonal lag, and 15 hidden layer neurons. This is a highly nonlinear model and it performs exceptionally well. It is the cleanest fit so far and there is nothing left in the residuals, just white noise.

```
nnet_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 nnet      13.3     0.500
```

A Ljung-Box test on the residuals confidently rejects the null that residuals are correlated. This indicates there are few present, and the NNETAR model is a good fit.

ARFIMA

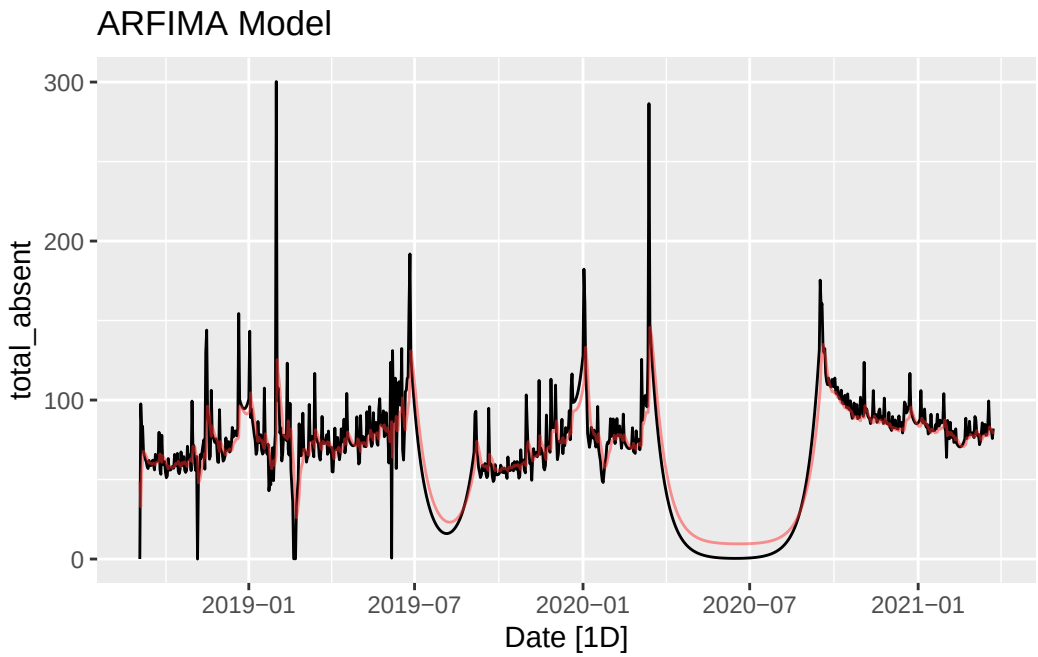
```
arfima_fit <- train_ts_attend |>
    model(
        arfima = ARFIMA(total_absent)
    )
```

```
Warning in sqrt(diag(best$var.coef)): NaNs produced
```

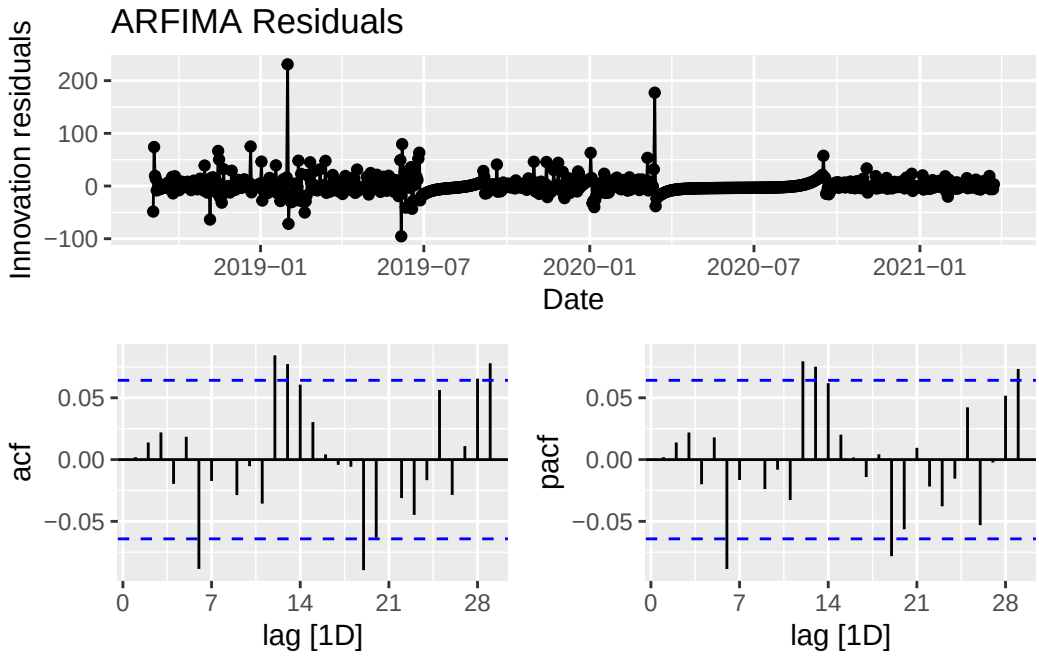
```
report(arfima_fit)
```

```
Series: total_absent
Model: ARFIMA(2,0.19,3)

Coefficients:
          mu      d     ar1     ar2      ma1      ma2     ma3
      65.051  0.191  0.5967  0.3449  -0.3432  -0.2503  0.0053
s.e.      NA  0.024     NaN     NaN      NaN      NaN  0.0106

sigma^2 estimated as 275.3:  log likelihood=-3942.54
AIC=7897.08   AICc=7897.17   BIC=7926.11
```

```
# fit plot
arfima_afit <- augment(arfima_fit)
train_ts_attend |>
    autoplot(total_absent) +
    geom_line(
        aes(x = Date, y = .fitted),
        arfima_afit,
        color = "firebrick1",
        alpha = 0.5) +
    labs(title = "ARFIMA Model")
```




```
# residuals plot
arfima_fit |>
    gg_tsresiduals(, plot_type = "partial") +
    labs(title = "ARFIMA Residuals")
```



Given the long memory of the series, ARFIMA should be a well performing model. The stepwise process fits an ARFIMA(2,0.19,3) model which, interestingly, has an extra MA term compared to the intial ARIMA model. The full difference might have obscured some MA dynamics that we are now capturing with the 0.19 difference. The fit looks good but a little bit lagged behind the data. The residuals resemble white noise, confirming a good fit.

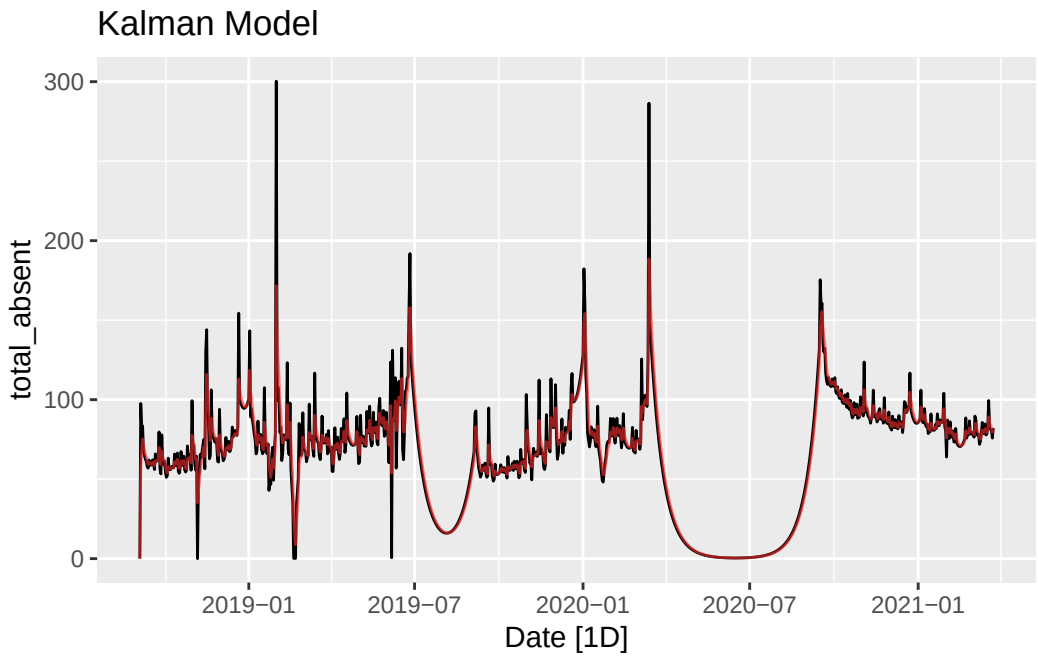
```
arfima_afit |>
    features(.innov, ljung_box, lag = 14)
```

```
# A tibble: 1 x 3
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 arfima    26.9    0.0199
```

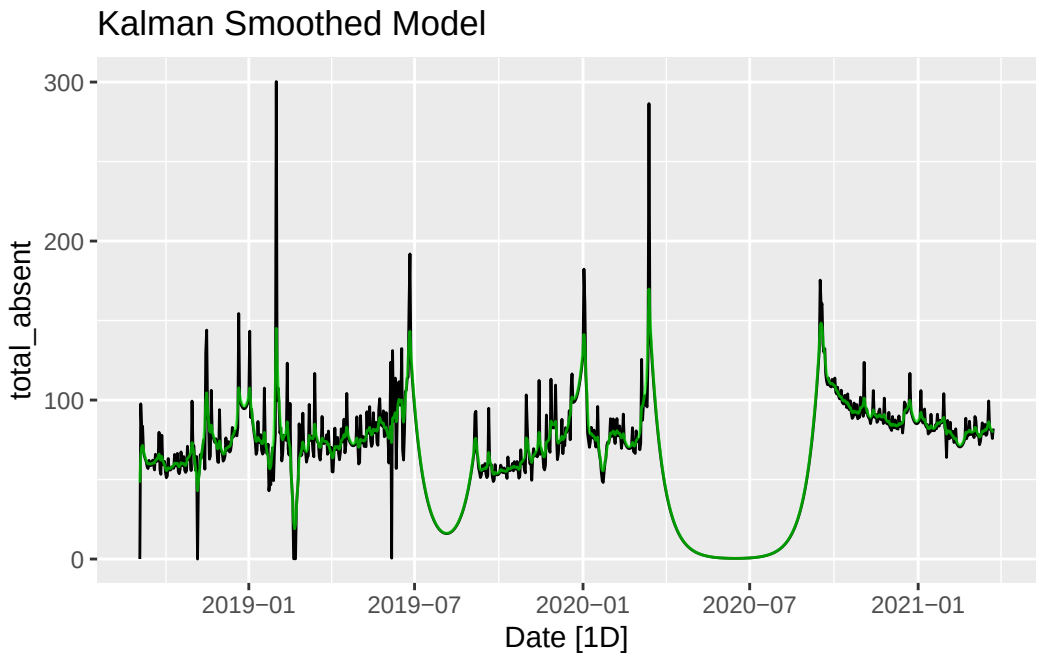
A Ljung-Box test on the residuals fails to reject the null that residuals are correlated. This indicates there are still dynaics present, and the ARFIMA model is not a perfect fit on its own.

Kalman Filter

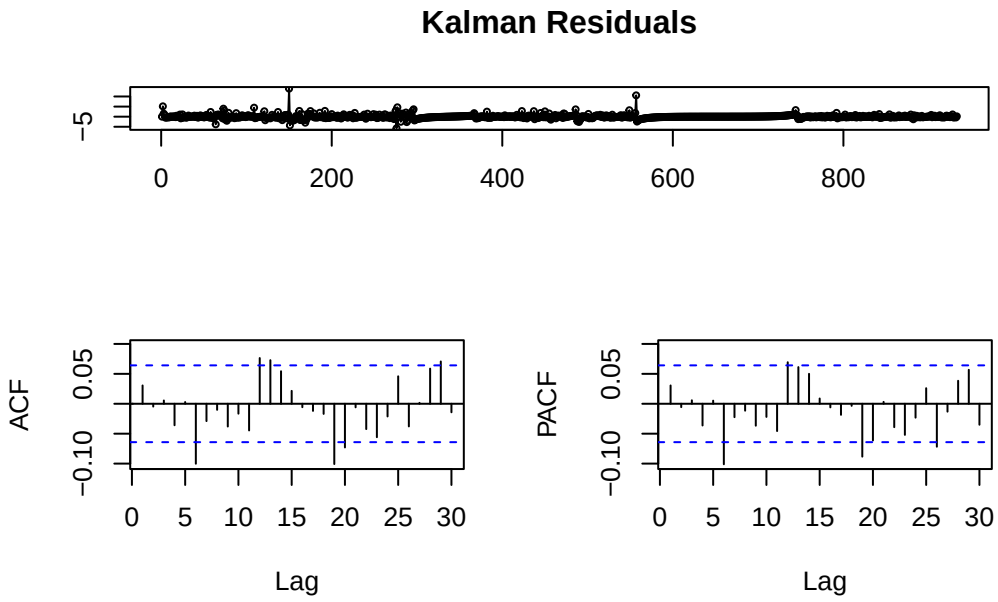
```
kalman_fit <- StructTS(train_ots_attend, type = "level")
kalman_fit_smooth <- tsSmooth(kalman_fit)

train_ts_attend |> autoplot(total_absent) +
    geom_line(
        aes(y = fitted(kalman_fit)),
        color = "firebrick1",
        alpha = 0.6
    ) +
    labs(title = "Kalman Model")
```




```
train_ts_attend |> autoplot(total_absent) +
    geom_line(
        aes(y = kalman_fit_smooth),
        color = "green",
        alpha = 0.6
    ) +
    labs(title = "Kalman Smoothed Model")
```




```
tsdisplay(kalman_fit$residuals, main = "Kalman Residuals")
```



The Kalman Filter's dynamic nature means it should perform exceptionally well on this series despite its odd structure and high frequency, and we can see that it does. Both the

regular and smoothed model are excellent fits to the data and the residuals are close to white noise.

```
Box.test(kalman_fit$residuals, type = "Ljung-Box", lag = 14)
```

```
    Box-Ljung test

data:  kalman_fit$residuals
X-squared = 29.364, df = 14, p-value = 0.009326
```

A Ljung-Box test on the residuals fails to reject the null that residuals are correlated. This indicates there are still some subtle dynamics present, and the Kalman Filter model alone is not a perfect fit.

Diagnostics

Now that I have an array of highly capable forecast models fitted, I run diagnostic tests to compare their performance on the testing set and develop a weighted forecast combination to use for my final analysis.

```
set.seed(144)

dummy_fc <- forecast(dummy_fit, h = 93)
arima_fc <- forecast(arima_fit, h = 93)
ets_fc <- forecast(ets_fit, h = 93)
hw_fc <- forecast(hw_fit, h = 93)
prophet_fc <- forecast(pro_fit, h = 93)
nnet_fc <- forecast(nnet_fit, h = 93) # slow
arfima_fc <- forecast(arfima_fit, h = 93)
kalman_fc <- forecast(kalman_fit, h = 93)

dummy_acc <- accuracy(dummy_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
arima_acc <- accuracy(arima_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
ets_acc <- accuracy(ets_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
hw_acc <- accuracy(hw_fc, test_ots_attend)[2, c(2, 3, 5)]
hw_acc <- data.frame(as.list(hw_acc))
pro_acc <- accuracy(prophet_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
nnet_acc <- accuracy(nnet_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
arfima_acc <- accuracy(arfima_fc, test_ts_attend)[c("RMSE", "MAE", "MAPE")]
kalman_acc <- accuracy(kalman_fc, test_ots_attend)[2, c(2, 3, 5)]
```

```r
kalman_acc <- data.frame(as.list(kalman_acc))

model_table <- dummy_acc |>
    full_join(arima_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(ets_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(hw_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(pro_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(nnet_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(arfima_acc, by = c("RMSE", "MAE", "MAPE")) |>
    full_join(kalman_acc, by = c("RMSE", "MAE", "MAPE"))

model_table <- model_table |>
    add_column("Model" = c(
        "Dummy",
        "ARIMA",
        "ETS",
        "Holt-Winters",
        "Prophet",
        "NNETAR",
        "ARFIMA",
        "Kalman"), .before = "RMSE")

knitr::kable(arrange(model_table, RMSE), row.names = FALSE)
```

Model	RMSE	MAE	MAPE
Holt-Winters	19.16604	10.74978	9.556811
NNETAR	20.10190	15.45606	15.370479
ETS	21.03257	12.54264	11.270009
Kalman	22.89052	14.33477	13.005006
ARIMA	22.90122	14.35168	13.023020
Dummy	28.05149	20.62807	19.600978
ARFIMA	30.37456	23.11499	22.123873
Prophet	60.98638	57.51473	59.830130

The models all perform quite well. The test set is less volatile than the training data because it does not include any breaks. This means many of the models actually perform better on the test set. Conveniently, the models outperform one anothe fairly cleanly on root mean squared error, mean absolute error, and mean absolute percent error. The definitive winner is the Holt-Winters model with RMSE: 21.0326, MAE: 12.5426, and MAPE: 11.27. Right behind that are the ETS, NNETAR, and Kalman Filter models. It seems the ETS fit was better than

anticipated when applied to the testing data. The worst performer is the prophet model, as anticipated, though it still performs decently well and may help the forecast combination.

Forecast Combination

I combine the seven model fits using inverse MSE weights that put more emphasis on the better performing models.

Inverse-MSE-Weights:

$$w_i = \frac{1/\text{MSE}_i^{\text{train}}}{\sum_j 1/\text{MSE}_j^{\text{train}}}, \qquad i \in \{\text{Dummy, ARIMA, ETS, HW, Prophet, NNETAR, ARFIMA, Kalman}\}$$

```
inv_mse <- function(acc){
    weight = (1/(acc$RMSE^2))/(sum(1/model_table$RMSE^2))
    return(weight)
}

weights <- c()
for (i in 1:7){
    weights <- append(weights, inv_mse(model_table[i,]))
}
names(weights) <- c("arima", "ets", "hw", "prophet", "nnet", "arfima", "kalman")
```

Since two of the models come from the `stats` package rather than `fable`, I compute the combination point forecast manually.

```
combination_fc <- (
    (weights["arima"]*(arima_fc$.mean)) +
    (weights["ets"]*(ets_fc$.mean)) +
    as.numeric(weights["hw"]*(hw_fc$mean)) +
    (weights["prophet"]*(prophet_fc$.mean)) +
    (weights["nnet"]*(nnet_fc$.mean)) +
    (weights["arfima"]*(arfima_fc$.mean)) +
    as.numeric(weights["kalman"]*(kalman_fc$mean))
)

combination_fc <- tibble("mean" = combination_fc, "Date" = test_ts_attend$Date)
combination_fc <- as.ts(combination_fc)

combination_fc |>
    accuracy(test_ots_attend)
```

```
                ME     RMSE      MAE     MPE     MAPE
Test set 33.83658 38.42553 33.91556 33.9126 34.05989
```

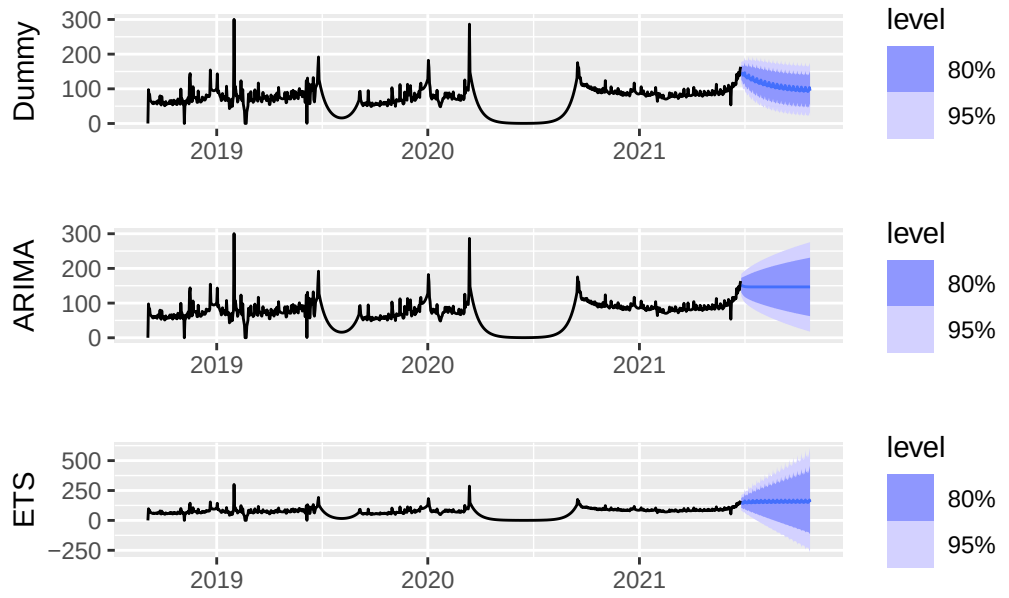
The combined training forecast performs decently, coming in around the middle of all the models on RMSE, MAE, and MAPE. I consider this the best fit to the true processs of the data and will now fit a final model to the full dataset for a true forecast.

Forecast

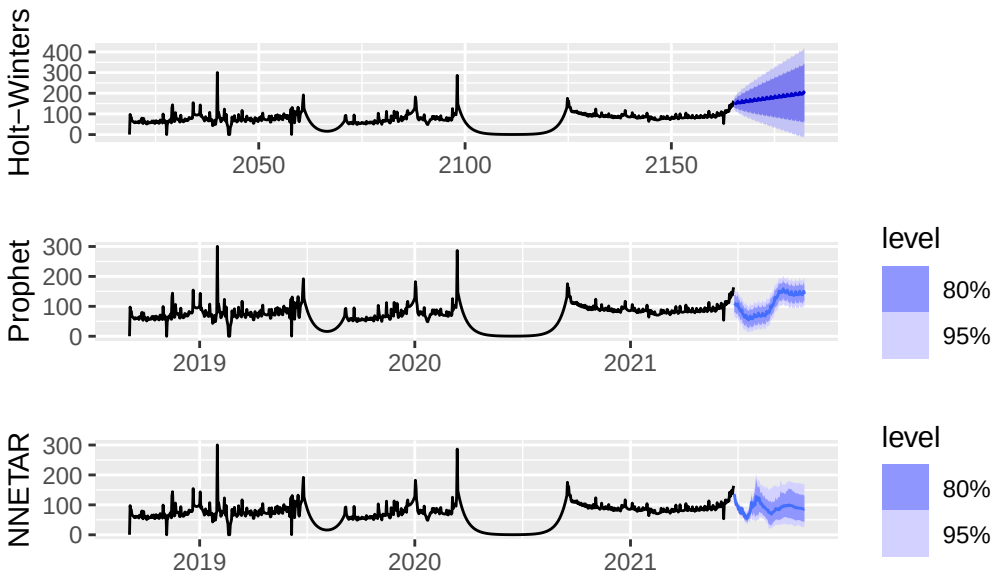
```
set.seed(144)

# refitting to full dataset
final_dummy_fit <- dummy_fit |>
    refit(ts_attend, reestimate = TRUE)
final_arima_fit <- arima_fit |>
    refit(ts_attend, reestimate = TRUE)
final_ets_fit <- ets_fit |>
    refit(ts_attend, reestimate = TRUE)
final_prophet_fit <- ts_attend |>
    model(prophet = pro_fit$prophet[[1]]$model) # refit not defined
final_nnet_fit <- nnet_fit |>
    refit(ts_attend, reestimate = TRUE)
final_arfima_fit <- arfima_fit |>
    refit(ts_attend, reestimate = TRUE)
final_hw_fit <- HoltWinters(ots_attend,
                        alpha = 0.4625732,
                        beta = 0.005831283,
                        gamma = 0.02644665,
                        seasonal = "additive")
final_kalman_fit <- tsSmooth(kalman_fit, ts_attend)
```

```
dummy_fc <- forecast(final_dummy_fit, h = 120)
arima_fc <- forecast(final_arima_fit, h = 120)
ets_fc <- forecast(final_ets_fit, h = 120)
hw_fc <- forecast(final_hw_fit, h = 120)
prophet_fc <- forecast(final_prophet_fit, h = 120)
nnet_fc <- forecast(final_nnet_fit, h = 120) # slow
arfima_fc <- forecast(final_arfima_fit, h = 120)
kalman_fc <- forecast(final_kalman_fit, h = 120)
```

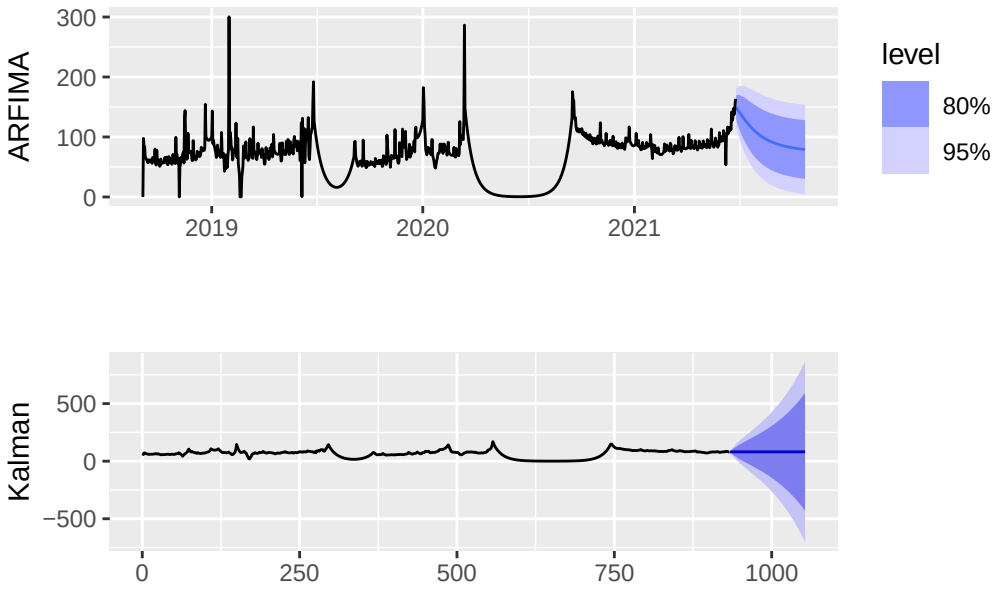
```
(autoplot(dummy_fc, ts_attend) + labs(y = "Dummy", x = "")) /
    (autoplot(arima_fc, ts_attend) + labs(y = "ARIMA", x = "")) /
    (autoplot(ets_fc, ts_attend) + labs(y = "ETS", x = ""))
```




```
(autoplot(hw_fc) + labs(title = "", y = "Holt-Winters", x = "")) /
    (autoplot(prophet_fc, ts_attend) + labs(y = "Prophet", x = "")) /
    (autoplot(nnet_fc, ts_attend) + labs(y = "NNETAR", x = ""))
```

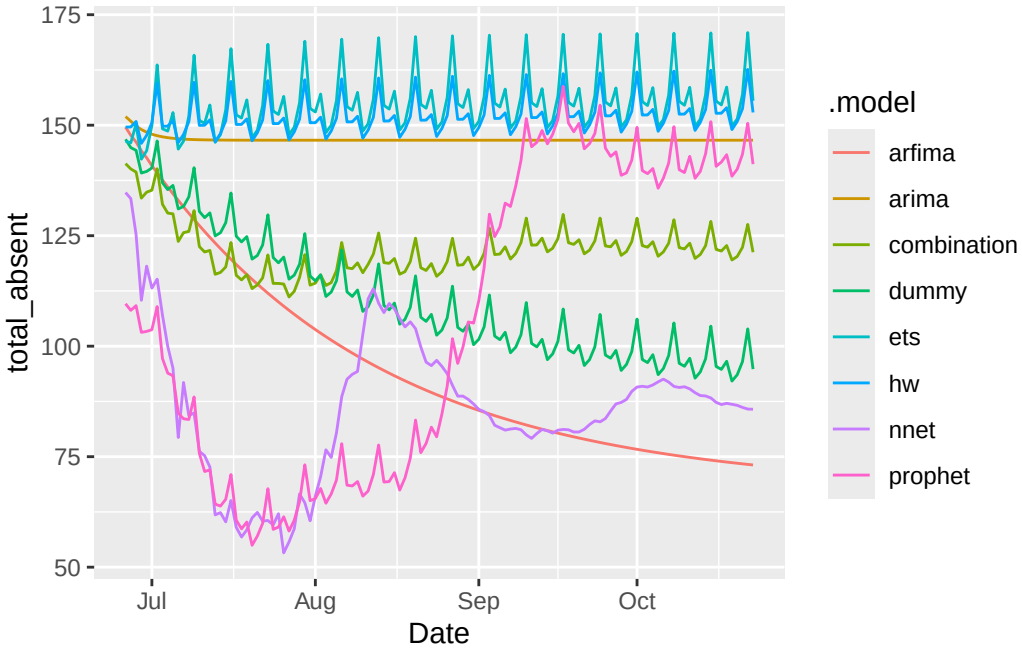



```
(autoplot(arfima_fc, ts_attend) + labs(y = "ARFIMA", x = "")) /
    (autoplot(kalman_fc) + labs(title = "", y = "Kalman", x = ""))
```



The final models ended up with a wide range of different predictions. Ideally, this means the combined forecast will benefit from substantial diversification, but it is worth considering that some of the models are quite far off. Intuitively we know that since the series ends in June (then end of the school year), future observations should tend toward zero for 1-2 months and then jump back up to the mean. I want to highlight the Kalman Filter which immediately reverts to the mean and gains major uncertainty once we stop feeding it new measurements. A model like this would be much more useful for administrators live forecasting attendance on a day to day basis. Overall, this is a difficult pattern to capture but some of the models appear to have learned it. I am pushing their limits with the 120 day forecast horizon and a few models revert to the mean immediately but overall they appear to be representing the series well. In the case of an individual school or community, forecasts like this made during an single school year could provide meaningful insight, without having to account for large summer dips. To help visualize the individual component forecasts and the series dynamics better, I also plot the set of fable models separately with a simple equal-weight combination.

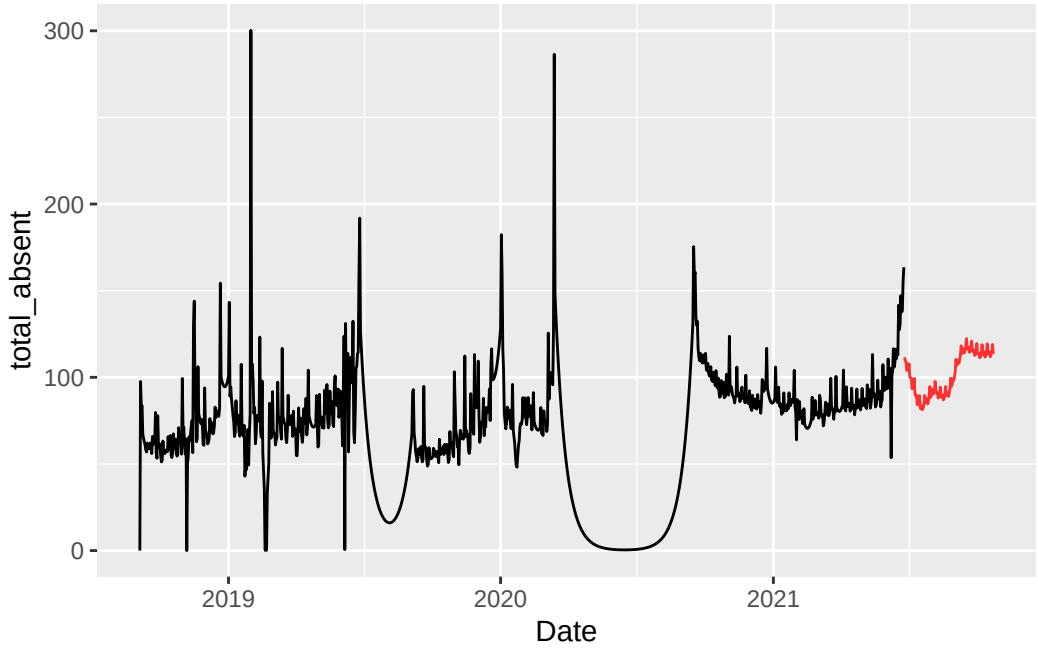
```
set.seed(144)

fable_fit <- ts_attend |>
    model(
        dummy = dummy_fit$dummy[[1]]$model,
        arima = arima_fit$arima[[1]]$model,
        ets = ets_fit$ets[[1]]$model,
        hw = ETS(total_absent ~ error("A") + trend("A") + season("A")),
        prophet = pro_fit$prophet[[1]]$model,
        nnet = nnet_fit$nnet[[1]]$model,
        arfima = arfima_fit$arfima[[1]]$model
    )

fable_fc <- fable_fit |>
    mutate(
        combination = (dummy + arima + ets + hw + prophet + nnet + arfima) / 7
    ) |>
    forecast(h = 120)

fable_fc |>
    autoplot(level = FALSE)
```



Looking at the different fable forecasts, we can see that ETS, a separate Holt-Winters estimate, and ARIMA all stay close to the last observed value, which is around the school year mean. Meanwhile Prophet and the Neural Network resemble what we expect the pattern to be; they fall in July and August before quickly climbing back up in September when school is back in session. ARFIMA drops as well but it may have put too much weight on the prolonged attendance lull due to the COVID-19 pandemic because it does not jump back up at the school year start. This is, admittedly, a difficult point to forecast from. Most of the models capture a distinctive weekly seasonality. The combination looks to be a good balance of all the fits, but I tune the final forecast to add more weight to the models that dipped during the summer. School administrators hoping to model attendance in this way could also include proprietary tuning or confounding variables like local events in their models.

```
# manually tuned based on outside information
# took 0.05 away from all other forecasts and gave to prophet and nnet
combination_fc <- (
    ((weights["arima"]-0.05)*(arima_fc$.mean)) +
    ((weights["ets"]-0.05)*(ets_fc$.mean)) +
    as.numeric((weights["hw"]-0.05)*(hw_fc$mean)) +
    ((weights["prophet"]+0.15)*(prophet_fc$.mean)) +
    ((weights["nnet"]+0.1)*(nnet_fc$.mean)) +
    ((weights["arfima"]-0.05)*(arfima_fc$.mean)) +
    as.numeric((weights["kalman"]-0.05)*(kalman_fc$mean))
)
```

```r
combination_fc <- tibble(
    "mean" = combination_fc,
    "Date" = seq(ymd("2021-06-26"), ymd("2021-10-23"), by = "days")
)
combination_fc <- tsibble(combination_fc, index = Date)

ggplot() +
    geom_line(data = ts_attend, aes(x = Date, y = total_absent)) +
    geom_line(data = combination_fc, aes(x = Date, y = mean), color = "firebrick1")
```



The final forecast captures the weekly seasonality and shows some recognition of the fact that their should be a dip and return to mean around the summer season. A prediction like this would be much more valuable during the school year but this is a notable showcase of the capability of advanced economic forecasting models to deal witih unusual dynamics.

III. Conclusions and Future Work

While this final forecast alone has limited applicability because it starts at the end of a school year. This report demonstrates an effective selection from the many potential forecast models that could be used by school officials to gain insight into the hidden dynamics in their attendance. Simpler analysis performed on any one of the thousands of individual schools in

this dataset could reveal fascinating details about local absenteeism trends and pave the way for targeted interventions that aim to counteract these dynamic patterns. Likewise, these same forecast models, and many more, could be applied to a wide array of issues in education and give meaningful insight into otherwise unknowable future trends. With access to proprietary data at higher frequencies, there is major potential for economic forecasting to improve day-to-day school operations.

Future work should dive deeper into the individual schools and communities represented in this dataset to try and uncover more concrete dynamics that can be understood in their local context. Models could also be created for the school year only, potentially training on the 2018-19 and 2019-20 school years then testing on the 2020-21 school year directly to avoid the unwanted influence of summer break. Models fit under that framework could be incredibly insightful.

IV. References

Data retrieved from: https://data.cityofnewyork.us/Education/2018-2021-Daily-Attendance-by-School/xc44-2jrh/about_data on June 4th, 2026